

14장 파일 입출력

1. 파일과 스트림의 이해

- * **파일의 개념**: 자료를 보조 기억장치에 저장하기 위한 특별한 형식으로, 데이터 바이트들이 순차적으로 저장되며 마지막에는 EOF(End of File) 표시가 있습니다. 내부적으로는 파일 내에서 현재 자료를 읽고 쓸 위치를 알려주는 ****파일 위치 지시자(file position indicator)****가 존재합니다.
- * **스트림(Stream)**: 입출력 장치와 프로그램을 연결해주는 채널로, 데이터의 연속된 바이트 흐름을 의미합니다. 운영체제가 자동으로 생성/관리하며, 대표적으로 ****표준 입력(stdin)**, **표준 출력(stdout)**, **표준 오류(stderr)****가 있습니다.

2. 파일 열기와 닫기

파일의 자료를 다루기 위해서는 먼저 파일 포인터(**FILE ***)를 통해 파일 스트림을 열어야 합니다.

- * **fopen()**, **fopen_s()**: 지정한 파일을 읽기(**r**), 쓰기(**w**), 추가(**a**) 등의 특정 접근 모드로 열고 **FILE** 구조체 포인터와 연결합니다.
- * **freopen()**: 이미 열려있는 스트림을 새로운 파일로 재연결하거나 파일 모드를 변경할 때 사용합니다.
- * **fclose()**: 입출력 작업이 끝나면 호출하며, 열려 있는 스트림과 파일의 연결을 해제하고 버퍼의 내용을 디스크에 안전하게 저장합니다.

파일 열기 모드 (File Open Modes)

모드	처리 내용	파일이 없는 경우	파일이 있는 경우
"r"	텍스트 파일 읽기(read)	NULL 반환	정상 처리
"w"	텍스트 파일 저장(write) 하기	새로운 파일 생성	이전 내용 삭제
"a"	텍스트 파일 추가(append)	새로운 파일 생성	이전 내용 뒤에 추가
"r+"	텍스트 파일 읽고 저장하기	NULL 반환	정상 처리
"w+"	텍스트 파일 읽고 저장하기	새로운 파일 생성	이전 내용 삭제

"a+"	텍스트 파일 추가로 읽고 저장	새로운 파일 생성	이전 내용 뒤에 추가
"rb"	2진(Binary) 파일 읽기	NULL 반환	정상 처리
"wb"	2진(Binary) 파일 저장 하기	새로운 파일 생성	이전 내용 삭제
"ab"	2진(Binary) 파일 추가	새로운 파일 생성	이전 내용 뒤에 추가
"r+b"	2진(Binary) 파일 읽고 저장하기	NULL 반환	정상 처리
"w+b"	2진(Binary) 파일 읽고 저장하기	새로운 파일 생성	이전 내용 삭제
"a+b"	2진(Binary) 파일 추가로 읽고 저장하기	새로운 파일 생성	이전 내용 뒤에 추가

3. 텍스트 파일 입출력

텍스트(표시용 문자와 줄바꿈 문자) 형식으로 구성된 파일을 처리하는 함수들입니다.

* 문자 단위 입출력:

- o `fgetc(fp)`: 파일로부터 한 문자를 읽어 반환합니다 (파일의 끝이거나 오류 시 EOF 반환).
- o `fputc(ch, fp)`: 파일에 단일 문자를 출력합니다.

* 문자열 단위 입출력:

- o `fgets(s, n, fp)`: 줄바꿈 문자나 파일 끝을 만날 때까지 최대 n-1개의 문자열을 버퍼로 읽어옵니다.
- o `fputs(s, fp)`: 버퍼의 문자열을 지정한 파일에 출력합니다.

* 형식화된 입출력:

- o `fscanf_s()`, `fprintf_s()` (또는 `fscanf`, `fprintf`): 서식 문자열(%d, %s 등)을 사용하여 다양한 자료형을 파일에 읽거나 쓰기 위한 함수로, 표준 입출력 함수인 `scanf`/`printf`와 사용법이 거의 동일합니다.

4. 이진 파일 입출력

텍스트 모드와 달리 데이터가 0과 1의 이진수(Binary) 형식으로 직접 저장되는 파일 처리 방식입니다. 문자로의 변환 과정이 없어 텍스트 방식보다 처리 속도가 빠르고 데이터 손실을 막을 수 있습니다.

- * 파일 열기 시 "rb", "wb", "ab" 처럼 모드 문자 뒤에 b를 붙여야 합니다.
- * **블록 단위 입출력:**
 - o `fread(buffer, size, count, fp)`: 지정한 크기(size)의 항목을 count개만큼 메모리로 통째로 읽어옵니다.
 - o `fwrite(buffer, size, count, fp)`: 버퍼에 있는 메모리 블록을 파일에 한 번에 출력합니다.

5. 파일 임의 접근 (Random Access)

순차적으로 파일에 접근하는 방식을 넘어, **파일 내 원하는 위치로 바로 이동하여 읽기/쓰기**를 수행하는 방식입니다.

- * `fseek(fp, offset, origin)`: 파일의 기준 위치(origin)로부터 지정된 바이트(offset)만큼 위치 지시자를 이동시킵니다. 기준 위치는 파일 시작(`SEEK_SET`), 현재 위치(`SEEK_CUR`), 파일 끝(`SEEK_END`)으로 지정합니다.
- * `ftell(fp)`: 파일의 시작점에서부터 현재 파일 위치 지시자까지의 바이트 수(오프셋)를 반환합니다.
- * `rewind(fp)`: 파일 위치 지시자를 가장 처음(시작 위치)으로 되돌립니다.

6. 오류 처리 및 기타 기능

입출력 도중 예기치 못한 상황이나 오류가 발생했는지 검증하는 함수들입니다.

- * `feof(fp) / ferror(fp)`: 파일의 끝(EOF)에 도달했는지, 또는 작업 중 물리적인 오류가 발생했는지를 확인합니다.
- * `strerror_s()`: 발생한 오류 번호(error code)를 인계받아 그에 해당하는 **시스템 오류 메시지 문자열**로 매핑해 줍니다.
- * **파일 관리 보조 함수**: 특정 파일을 삭제하는 `remove()`, 파일 이름을 변경하는 `rename()`, 그리고 스트림 내부 버퍼를 수동으로 비우는 `fflush()`가 있습니다.